



Bangladesh University of Business and Technology

Lab report no : 1

Lab topic: List ,Tuples, Dictionary

Course name :Data Mining Lab

Course code: CSE 476

Submitted by

Name: Md.Al-Amin

Id: 22234103066

Section : 5

Intake : 50

Submitted to

Md.Aminul Islam Rafi

Lecturer

Dept of CSE ,BUBT

Python List:

List is one of the built-in data types in Python. A Python list is a sequence of comma separated items, enclosed in square brackets []. The items in a Python list need not be of the same data type.

List is an ordered collection of items. Each item in a list has a unique position index, starting from 0.

```
listA = ["apple", "mango", 10.4, 75]
listB = [30, 80, 130, 240, 550]
listC = ["P", "N", "M", "L"]
listD = [25.50, True, -55, 4+2j]
```

1.Accessing value in list

```
: listA = ["apple", "mango", 10.4, 75]
listB = [30, 80, 130, 240, 550]
print ("listA[0]: ", listA[0]) #accessing index 0
print ("listB[1:5]: ", listB[1:5]) #accessing index 1 to 4

listA[0]: apple
listB[1:5]: [80, 130, 240, 550]
```

2.Update list

```
{]: listA = ["apple", "mango", 10.4, 75]
print ("Value available at index 2 : ")
print (listA[2])
listA[2] = 200;
print ("New value available at index 2 : ")
print (listA[2])

Value available at index 2 :
10.4
New value available at index 2 :
200
```

3.Delete list elements

```
[15]: listA = ["apple", "mango", 10.4, 75]
print (listA)
del listA[2];
print ("After deleting value at index 2 : ")
print (listA)

['apple', 'mango', 10.4, 75]
After deleting value at index 2 :
['apple', 'mango', 75]
```

4.list concatenation

```
] a = ['x', 'y']  
  b = ['z']  
  print(a + b)
```

5.Repetition

```
] print(['1'] * 3)  
  
['1', '1', '1']
```

6.Indexing,Slicing

```
L = ['A', 'B', 'C', 'D']  
print(L[2])  
print(L[-2])  
print(L[1:])
```

```
C  
C  
['B', 'C', 'D']
```

Python list methods

```
# Initial list  
my_list = [3, 1, 4, 1, 5, 9]  
print("Initial list:", my_list)
```

Initial list: [3, 1, 4, 1, 5, 9]

1.list.append()

```
# 1. append(obj): Adds an item at the end  
my_list.append(2)  
print("After append(2):", my_list)
```

After append(2): [3, 1, 4, 1, 5, 9, 2]

2.list.clear() and list.copy()

```
# 2. clear(): Removes all items  
# 3. copy(): Make a copy before clearing  
temp_list = my_list.copy() # save copy  
my_list.clear()  
print("After clear():", my_list)  
print("Temp copy for later use:", temp_list)
```

After clear(): []
Temp copy for later use: [3, 1, 4, 1, 5, 9, 2]

3.list.copy()

```
# 3. copy(): again, to demonstrate
copied_list = temp_list.copy()
print("Copied list:", copied_list)
```

Copied list: [3, 1, 4, 1, 5, 9, 2]

4.list.count(obj)

```
9]: # 4. count(obj): Counts occurrences
count_1 = copied_list.count(1)
print("Count of 1:", count_1)
```

Count of 1: 2

5.list.extend(seq)

```
# 5. extend(seq): Adds all elements of a sequence
copied_list.extend([6, 5])
print("After extend([6, 5]):", copied_list)
```

After extend([6, 5]): [3, 1, 4, 1, 5, 9, 2, 6, 5]

6.list.index(obj)

```
# 6. index(obj): First index of a value
index_5 = copied_list.index(5)
print("Index of first occurrence of 5:", index_5)
```

7.list.insert(index,obj)

```
: # 6. insert(index, obj): Insert value at position
copied_list.insert(0, 0)
print("After insert(0, 0):", copied_list)
```

After insert(0, 0): [0, 3, 1, 4, 1, 5, 9, 2, 6, 5]

8.list.reverse()

```
# 10. reverse(): Reverses list in place
copied_list.reverse()
print("After reverse():", copied_list)
```

After reverse(): [5, 6, 2, 9, 5, 1, 4, 1, 3, 0]

9.list.sort()

```
# sort(): Sorts list in place
copied_list.sort()
print("After sort():", copied_list)
```

After sort(): [0, 1, 1, 2, 3, 4, 5, 5, 6, 9]

Built in function

```
list1 = [1, 2, 3, 4, 5, 6, 7 ];
my_tuple = (10, 20, 30)
print("Length of list1:", len(list1))
print("Maximum value in list1:", max(list1))
print("Minimum value in list1:", min(list1))
converted_list = list(my_tuple)
print("Converted tuple to list:", converted_list)
```

Length of list1: 7
Maximum value in list1: 7
Minimum value in list1: 1
Converted tuple to list: [10, 20, 30]

Loop list

```
list4 = [25, 12, 10, -21, 10, 100]
for num in list4:
    print (num, end = ' ')
```

25 12 10 -21 10 100

Tuples

Tuple is one of the built-in data types in Python. A Python tuple is a sequence of comma separated items, enclosed in parentheses (). The items in a Python tuple need not be of same data type.

```
: tup1 = ("Rohan", "Physics", 21, 69.75)
   tup2 = (1, 2, 3, 4, 5)
   tup3 = ("a", "b", "c", "d")
   tup4 = (25.50, True, -55, 1+2j)
```

Accessing value in tuple

```
tup1 = ('physics', 'chemistry', 1997, 2000);
tup2 = (1, 2, 3, 4, 5, 6, 7 );
print ("tup1[0]: ", tup1[0]);
print ("tup2[1:5]: ", tup2[1:5]);
```

tup1[0]: physics
tup2[1:5]: (2, 3, 4, 5)

Updating tuples

Following action is not valid for tuples

```
tup1[0] = 100;
```

```
tup1 = (12, 34.56);
tup2 = ('abc', 'xyz');

# Following action is not valid for tuples
# tup1[0] = 100;

# So let's create a new tuple as follows
tup3 = tup1 + tup2;
print (tup3);
```

```
(12, 34.56, 'abc', 'xyz')
```

Delete Tuple elements

```
tup = ('physics', 'chemistry', 1997, 2000);
print (tup);
del tup;
print ("After deleting tup : ");
print (tup);
```

```
('physics', 'chemistry', 1997, 2000)
After deleting tup :
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[71], line 5
      3 del tup;
      4 print ("After deleting tup : ");
----> 5 print (tup)

NameError: name 'tup' is not defined
```

Python Tuples operation:

Concatenation and repetition

```
#concatenation
tupA=(1,2,3)
tupB=(4,5,6)
tupC=tupA+tupB
print(tupC)

#Repetition
print((1,2,3,4)*2)
```

```
(1, 2, 3, 4, 5, 6)
(1, 2, 3, 4, 1, 2, 3, 4)
```

Indexing Slicing

```
# Define a tuple
L = ('spam', 'Spam', 'SPAM!')

# Indexing
print("L[2]:", L[2])      # Offsets start at zero → 'SPAM!'
print("L[-2]:", L[-2])    # Negative index → second from end → 'Spam'

# Slicing
print("L[1:]:", L[1:])    # Slicing from index 1 to end → ('Spam', 'SPAM!')
```

L[2]: SPAM!
L[-2]: Spam
L[1:]: ('Spam', 'SPAM!')

Built in function with tuples

```
# Built-in Functions with Tuples

# Sample tuple
my_tuple = (10, 25, 7, 33)

# 1. len(): Returns the number of elements in the tuple
print("Length of the tuple:", len(my_tuple))

# 2. max(): Returns the maximum value in the tuple
print("Maximum value in the tuple:", max(my_tuple))

# 3. min(): Returns the minimum value in the tuple
print("Minimum value in the tuple:", min(my_tuple))

# 4. tuple(): Converts a sequence (e.g., list) into a tuple
my_list = [1, 2, 3, 4]
converted_tuple = tuple(my_list)
print("Converted list to tuple:", converted_tuple)
```

Length of the tuple: 4
Maximum value in the tuple: 33
Minimum value in the tuple: 7
Converted list to tuple: (1, 2, 3, 4)

Looping tuples

```
tup = (25, 12, 10, -21, 10, 100)
for num in tup:
    print (num, end = ' ')
```

25 12 10 -21 10 100

Dictionaries in Python

In Python, a dictionary is a built-in data type that stores data in key-value pairs. It is an unordered, mutable, and indexed collection. Each key in a dictionary is unique and maps to a value. Dictionaries are often used to store data that is related, such as information associated with a specific entity or object, where you can quickly retrieve a value based on its key.

Python's dictionary is an example of a mapping type. A mapping object 'maps' the value of one object to another. To establish mapping between a key and a value, the colon (:) symbol is put between the two.

```
capitals = {"Maharashtra":"Mumbai", "Gujarat":"Gandhinagar", "Telangana":"Hyderabad", "Karnataka":"Bengaluru"}
numbers = {10:"Ten", 20:"Twenty", 30:"Thirty",40:"Forty"}
marks = {"Savita":67, "Imtiaz":88, "Laxman":91, "David":49}
print(capitals)
print(numbers)
print(marks)

{'Maharashtra': 'Mumbai', 'Gujarat': 'Gandhinagar', 'Telangana': 'Hyderabad', 'Karnataka': 'Bengaluru'}
{10: 'Ten', 20: 'Twenty', 30: 'Thirty', 40: 'Forty'}
{'Savita': 67, 'Imtiaz': 88, 'Laxman': 91, 'David': 49}
```

Creating dictionaries

```
# Creating a dictionary using curly braces
sports_player = {
    "Name": "Sachin Tendulkar",
    "Age": 48,
    "Sport": "Cricket"
}
print ("Dictionary using curly braces:", sports_player)
# Creating a dictionary using the dict() function
student_info = dict(name="Alice", age=21, major="Computer Science")
print("Dictionary using dict():",student_info)

Dictionary using curly braces: {'Name': 'Sachin Tendulkar', 'Age': 48, 'Sport': 'Cricket'}
Dictionary using dict(): {'name': 'Alice', 'age': 21, 'major': 'Computer Science'}
```

Accessing dictionary item

```
student_info = {
    "name": "Alice",
    "age": 21,
    "major": "Computer Science"
}
# Accessing values using square brackets
name = student_info["name"]
print("Name:",name)

# Accessing values using the get() method
age = student_info.get("age")
print("Age:",age)

Name: Alice
Age: 21
```


Modify dictionary item

```
student_info = {  
    "name": "Alice",  
    "age": 21,  
    "major": "Computer Science"  
}  
# Modifying an existing key-value pair  
student_info["age"] = 22  
  
# Adding a new key-value pair  
student_info["graduation_year"] = 2023  
print("The modified dictionary is:", student_info)
```

The modified dictionary is: {'name': 'Alice', 'age': 22, 'major': 'Computer Science', 'graduation_year': 2023}

Removing dictionary item

```
student_info = {  
    "name": "Alice",  
    "age": 22,  
    "major": "Computer Science",  
    "graduation_year": 2023  
}  
# Removing an item using the del statement  
del student_info["major"]  
  
# Removing an item using the pop() method  
graduation_year = student_info.pop("graduation_year")  
  
print(student_info)
```

{'name': 'Alice', 'age': 22}

Iterating throw dictionary

```
student_info = {  
    "name": "Alice",  
    "age": 22,  
    "major": "Computer Science",  
    "graduation_year": 2023  
}  
  
# Iterating through keys  
for key in student_info:  
    print("Keys:",key, student_info[key])  
  
# Iterating through values  
for value in student_info.values():  
    print("Values:",value)  
  
# Iterating through key-value pairs  
for key, value in student_info.items():  
    print("Key:Value:",key, value)
```

```
Keys: name Alice  
Keys: age 22  
Keys: major Computer Science  
Keys: graduation_year 2023  
Values: Alice  
Values: 22  
Values: Computer Science  
Values: 2023  
Key:Value: name Alice  
Key:Value: age 22  
Key:Value: major Computer Science  
Key:Value: graduation_year 2023
```

Properties of dictionary

Dictionary values have no restrictions. They can be any arbitrary Python object, either standard objects or user-defined objects. However, same is not true for the keys.

There are two important points to remember about dictionary keys –

More than one entry per key not allowed. Which means no duplicate key is allowed. When duplicate keys encountered during assignment, the last assignment wins. For example –

```
dict = {'Name': 'Zara', 'Age': 7, 'Name': 'Manni'}  
print ("dict['Name']: ", dict['Name'])  
  
dict['Name']: Manni
```

keys must be immutable. Which means you can use strings, numbers or tuples as dictionary keys but something like ['key'] is not allowed. Following is a simple example

```
dict = {'Name': 'Zara', 'Age': 7}
print ("dict['Name']: ", dict['Name'])
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[107], line 1
----> 1 dict = {'Name': 'Zara', 'Age': 7}
      2 print ("dict['Name']: ", dict['Name'])

TypeError: unhashable type: 'list'
```

Python dictionary operators

```
# Define two dictionaries
d1 = {'a': 2, 'b': 4, 'c': 30}
d2 = {'a1': 20, 'b1': 40, 'c1': 60}

# Access value using key
print("Value of d1['b']:", d1['b']) # Extracts 4

# Assign a new value to key 'b'
d1['b'] = 'Z'
print("After assignment, d1:", d1) # Now 'b' has value 'Z'

# Dictionary union using |
d3 = d1 | d2
print("Union of d1 and d2 using '|':", d3)

# Augmented union using |=
d1 |= d2
print("After d1 |= d2, d1 becomes:", d1)

Value of d1['b']: 4
After assignment, d1: {'a': 2, 'b': 'Z', 'c': 30}
Union of d1 and d2 using '|': {'a': 2, 'b': 'Z', 'c': 30, 'a1': 20, 'b1': 40, 'c1': 60}
After d1 |= d2, d1 becomes: {'a': 2, 'b': 'Z', 'c': 30, 'a1': 20, 'b1': 40, 'c1': 60}
```

Dictionary methods

```
# Sample dictionary
d = {'name': 'Alice', 'age': 25, 'city': 'Dhaka'}

# 1. clear(): Removes all elements
d_copy = d.copy()      # Save copy before clearing
d.clear()
print("After clear():", d)

# 2. copy(): Creates a shallow copy
print("Copy of original dict:", d_copy)

# 3. fromkeys(): Creates a new dict from keys with default value
keys = ['a', 'b', 'c']
new_dict = dict.fromkeys(keys, 0)
print("New dict from keys with default 0:", new_dict)

# 4. get(): Returns value if key exists, else default
print("Get 'name':", d_copy.get('name'))      # Output: Alice
print("Get 'gender' with default:", d_copy.get('gender', 'Not found'))
```

After clear(): {}
Copy of original dict: {'name': 'Alice', 'age': 25, 'city': 'Dhaka'}
New dict from keys with default 0: {'a': 0, 'b': 0, 'c': 0}
Get 'name': Alice
Get 'gender' with default: Not found

```
# Reusing d_copy = {'name': 'Alice', 'age': 25, 'city': 'Dhaka'}

# 6. items(): Returns key-value pairs
print("Items:", d_copy.items())

# 7. keys(): Returns keys of dictionary
print("Keys:", d_copy.keys())

# 8. setdefault(): Gets value or sets default if key doesn't exist
d_copy.setdefault('gender', 'Female')
print("After setdefault('gender', 'Female'):", d_copy)

# 9. update(): Merges another dictionary
d_copy.update({'age': 26, 'country': 'Bangladesh'})
print("After update:", d_copy)

# 10. values(): Returns values of dictionary
print("Values:", d_copy.values())
```

Items: dict_items([('name', 'Alice'), ('age', 25), ('city', 'Dhaka')])
Keys: dict_keys(['name', 'age', 'city'])
After setdefault('gender', 'Female'): {'name': 'Alice', 'age': 25, 'city': 'Dhaka', 'gender': 'Female'}
After update: {'name': 'Alice', 'age': 26, 'city': 'Dhaka', 'gender': 'Female', 'country': 'Bangladesh'}
Values: dict_values(['Alice', 26, 'Dhaka', 'Female', 'Bangladesh'])

Built in functions in dictionary

```
# Define a sample dictionary
my_dict = {'name': 'Alice', 'age': 25, 'city': 'Dhaka'}

# 1. len(): Returns the number of key-value pairs in the dictionary
print("Length of dictionary:", len(my_dict)) # Output: 3

# 2. str(): Returns a string representation of the dictionary
print("String representation of dictionary:", str(my_dict))

# 3. type(): Returns the type of the variable
print("Type of my_dict:", type(my_dict)) # Output: <class 'dict'>
```

Length of dictionary: 3

String representation of dictionary: {'name': 'Alice', 'age': 25, 'city': 'Dhaka'}

Type of my_dict: <class 'dict'>